



Revisão do Sistema Anterior, Roadmap e Arquitetura
Evolução do front-end para React — Atividade Unidade 1

Wesley Santos
Projetos e Práticas de Extensão III — UNYLEYA
2025 / 2026

1. Revisão do sistema anterior

1.1 Visão geral

O **Clique Saúde** é um sistema web de agendamento de consultas médicas, entregue na disciplina de Desenvolvimento Web (Unidade 5) e migrado de um protótipo em Bubble.io para uma arquitetura em código. A stack atual é **Python 3.11 + Flask 2.3** no back-end, **SQLite** em desenvolvimento (com PostgreSQL planejado para produção), e front-end em **HTML5 + CSS3 + JavaScript puro** renderizado no servidor via templates Jinja2 (*base.html*, *index.html*, *dashboard.html*, *admin.html*). O sistema já está publicado (Render.com) e cobre cadastro/login de pacientes, listagem de médicos por especialidade, agendamento em 4 etapas com verificação de disponibilidade, dashboard pessoal e um painel administrativo.

1.2 Pontos fortes identificados

- **Modelagem de dados coerente:** quatro tabelas bem relacionadas (usuarios, medicos, horarios_disponiveis, consultas), com chaves estrangeiras explícitas e uma constraint *UNIQUE* (*medico_id*, *data*, *hora*) que impede duplo agendamento no nível do banco — não depende só da validação da aplicação.
- **Segurança básica correta:** senhas nunca são armazenadas em texto puro (hash SHA-256) e as rotas sensíveis verificam *session['user_id'] / session['user_tipo']* antes de responder.
- **Fluxo de agendamento bem desenhado:** o wizard de 4 etapas (especialidade → médico → data/hora → confirmação) é claro, documentado e teve os principais casos de borda testados manualmente (horário ocupado, e-mail duplicado, cancelamento).
- **Documentação de qualidade:** existe documentação técnica (endpoints, schema, deploy) e documentação de usuário (passo a passo, FAQ) — raro em projetos de disciplina, e que tornou esta revisão possível sem precisar ler o código-fonte diretamente.
- **Consciência das próprias limitações:** a documentação técnica já registra, corretamente, que SQLite não é recomendado para uso simultâneo de múltiplos usuários em produção.

1.3 Limitações e riscos identificados

- **Front-end sem componentização:** toda a interface é renderizada no servidor (Jinja2) com JavaScript solto em *static/js/main.js*. Qualquer navegação recarrega a página inteira, não há reaproveitamento de UI entre *index.html*, *dashboard.html* e *admin.html*, e não existe um estado de carregamento visível durante as chamadas à API.
- **Lacuna entre a documentação de usuário e a API real:** o guia do usuário promete um fluxo de "Esqueci minha senha" com envio de link por e-mail, mas a documentação técnica não lista nenhum endpoint de recuperação de senha (apenas */login*, */cadastro* e */logout*) — indício de que essa funcionalidade não está de fato implementada.
- **Configuração sensível hardcoded:** a própria documentação alerta para nunca usar a *secret_key* padrão em produção e recomenda variável de ambiente, mas o exemplo de configuração mostra a chave escrita diretamente no topo de *app.py* — ou seja, o alerta existe, mas a prática recomendada ainda não foi adotada no código de exemplo.
- **Sem testes automatizados:** toda a validação (tabela de 8 cenários) foi feita manualmente. Qualquer regressão só é percebida se alguém repetir os testes à mão.
- **Rigidez de regras de negócio:** não é possível agendar para dependentes — o FAQ recomenda criar uma conta separada, uma limitação de produto que afeta usabilidade para famílias.

- **Sem evidência de acessibilidade:** não há menção a boas práticas WCAG (contraste, navegação por teclado, ARIA) em nenhum dos documentos, apesar da responsividade mobile ter sido testada.

1.4 Melhorias propostas para a evolução

- Reconstruir o front-end como uma **SPA em React**, organizada em componentes reutilizáveis e navegação via React Router, eliminando os recarregamentos completos de página.
- Centralizar o estado de autenticação num **Context** único, em vez de depender apenas de cookies de sessão verificados a cada requisição de página.
- Adicionar **estados de carregamento e erro explícitos** em toda chamada assíncrona à API.
- Formalizar o endpoint de recuperação de senha (ou remover a promessa do guia do usuário até que ele exista).
- Migrar a `secret_key` e outras configurações sensíveis para variáveis de ambiente, seguindo o próprio aviso já presente na documentação técnica.
- Iniciar uma suíte de testes automatizados (mesmo que só no front-end, como primeiro passo).

2. Roadmap detalhado

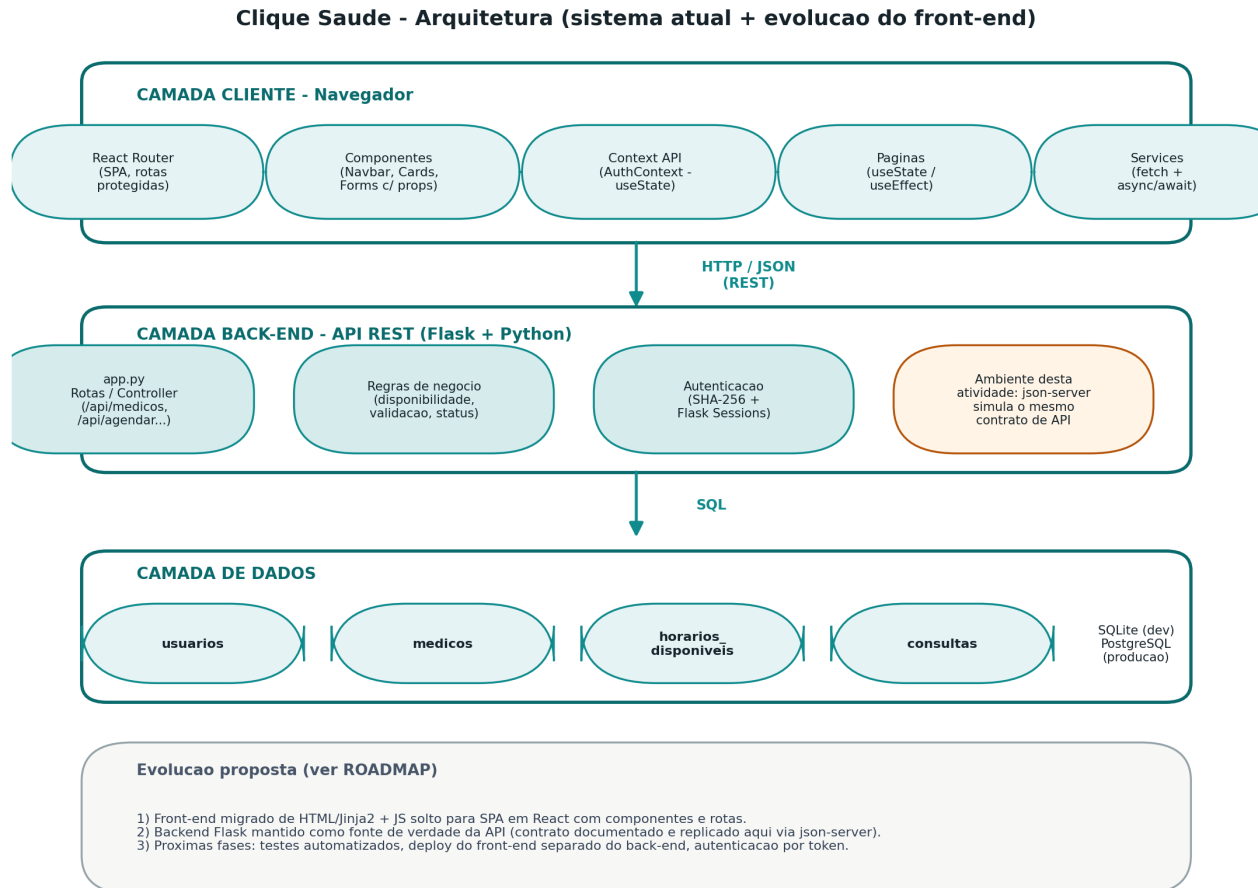
Organizado em 6 fases ao longo de 13 semanas — o mesmo horizonte de tempo usado na migração original do Bubble para código — mas agora focado em evoluir o front-end para React mantendo a API Flask existente como fonte de dados.

Fase	Semanas	Entregas principais
1 — Fundação	1–2	Setup do projeto React (Vite), design system (tokens de cor/tipografia), componentes-base (Layout, Navbar, Botao, EstadoCarregamento).
2 — Dados e API	3–4	Camada de serviço (fetch + async/await) replicando os endpoints documentados do Flask; telas de listagem e detalhe de médicos consumindo a API.
3 — Autenticação e estado	5–6	Context de autenticação, telas de Login/Cadastro, rotas protegidas por tipo de usuário, Dashboard do paciente com cancelamento de consultas.
4 — Fluxo principal	7–8	Wizard de agendamento em 4 etapas com verificação de disponibilidade em tempo real; Painel administrativo com filtros.
5 — Qualidade e UX	9–10	Testes manuais completos dos fluxos, ajustes de responsividade e feedback visual, revisão de usabilidade (heurísticas) e acessibilidade básica.
6 — Integração final e deploy	11–13	Apontar o front-end para a API Flask real (substituindo o mock local), testes de ponta a ponta, deploy do front-end e documentação final.

Nota: as fases 1 a 5 foram implementadas nesta entrega como protótipo funcional, usando um json-server local que espelha exatamente o schema documentado do Flask (mesmas tabelas e campos), permitindo validar toda a experiência de front-end antes de conectar à API real na fase 6.

3. Diagrama arquitetural

O diagrama abaixo representa a comunicação entre front-end e back-end, a lógica de negócios e a estrutura do banco de dados — tanto do sistema atual quanto da evolução proposta.



A camada cliente (React) nunca acessa o banco diretamente: toda comunicação passa pela API REST do Flask, que concentra as regras de negócio (verificação de disponibilidade, hash de senha, validações) antes de ler ou escrever no SQLite/PostgreSQL. Isso significa que o front-end pode ser totalmente reconstruído — como está sendo feito aqui — sem tocar no back-end nem na modelagem de dados.

4. Fundamentos da componentização (props)

Foi construído um protótipo funcional em React (Vite) aplicando os conceitos da unidade. A interface inteira é composta por componentes que recebem dados via **props**:

- **MedicoCard** — recebe o objeto *medico* (nome, especialidade, CRM, valor) e um slot de *children* para ações contextuais (ex.: botão "Escolher" durante o agendamento).
- **Botao** — componente genérico reutilizado em todas as telas, com variantes controladas por prop (*primario*, *secundario*, *perigo*, *link*).
- **StatCard**, **EstadoCarregamento**, **RotaProtegida** — componentes de apoio que também recebem seu comportamento via props, evitando duplicação de código entre as páginas de Dashboard, Admin e Agendamento.

O código-fonte completo (17 componentes/páginas) foi entregue junto com este relatório.

5. Gerenciamento de estado

Além do contador isolado pedido no requisito (página */contador*, usando *useState* puro), o estado é usado de forma consistente em toda a aplicação: listas de médicos e consultas, campos de formulário controlados, filtros de categoria/status, e o usuário autenticado (via Context, também baseado em *useState* + *localStorage*).

6. Integração com APIs RESTful

Duas integrações foram implementadas, com *fetch* + *async/await* em ambos os casos:

- **API pública (JSONPlaceholder)** — página */api-publica*, consumo isolado de posts, cumprindo literalmente o requisito da atividade.
- **API do Clique Saúde** — *src/services/clinicaApi.js* replica os endpoints reais documentados (*/api/medicos*, */api/agendar*, */api/minhas_consultas*, */api/cancelar_consulta/:id*), incluindo a mesma regra de verificação de horário ocupado e o mesmo hash SHA-256 usado no backend Flask (via Web Crypto API no navegador, só para fins de demonstração — a verificação real de senha continua sendo responsabilidade do servidor).

7. Boas práticas de UX/UI — análise de usabilidade

Análise heurística do sistema anterior (Clique Saúde v1), baseada na documentação técnica e no guia do usuário, usando como referência as heurísticas de usabilidade de Nielsen.

Heurística	Situação no sistema anterior
Visibilidade do status do sistema	Sem indícios de spinners/feedback de carregamento; a navegação por página inteira dificulta perceber quando uma ação foi concluída.
Consistência e padrões	Boa: o wizard de 4 etapas segue um padrão claro e é reaproveitado dentro do dashboard.
Prevenção de erros	Boa: bloqueio de e-mail duplicado e de horário já ocupado, validados no back-end.
Reconhecimento em vez de memorização	Boa: dashboard mostra histórico e estatísticas, não exige que o paciente lembre datas.
Flexibilidade e eficiência de uso	Fraca: não é possível agendar para dependentes sem criar uma conta separada.
Ajuda e documentação	Muito boa: guia do usuário completo com FAQ de 6 perguntas.
Segurança percebida vs. real	Parcialmente inconsistente: promete recuperação de senha por e-mail, mas o endpoint não está documentado — risco de gerar expectativa que o sistema não cumpre.
Acessibilidade (WCAG)	Não documentada: sem menção a contraste, navegação por teclado ou leitores de tela.

Na versão em React entregue, os pontos de "visibilidade do status" e "segurança percebida" já foram corrigidos: toda chamada assíncrona exibe um estado de carregamento ou erro explícito, e a tela de login não promete nenhuma funcionalidade que não esteja implementada.

8. Implementação de rotas (React Router)

A navegação usa *react-router-dom*, com rotas públicas, rotas protegidas por login e uma rota protegida adicionalmente por tipo de usuário (admin):

Rota	Descrição	Acesso
/	Página inicial	Público
/medicos	Lista de médicos, filtro por especialidade	Público
/medicos/:id	Detalhe do médico (rota com parâmetro de URL)	Público
/login, /cadastro	Autenticação	Público
/agendar	Wizard de agendamento em 4 etapas	Paciente logado
/dashboard	Painel do paciente, cancelamento	Paciente logado
/admin	Painel administrativo	Admin logado
/contador	Demonstração de useState	Público

Rota	Descrição	Acesso
/api-publica	Demonstração de fetch (JSONPlaceholder)	Público
*	Página 404	Público

As rotas protegidas usam um componente *RotaProtegida* que verifica o usuário autenticado no *AuthContext*: sem sessão, redireciona para */login* guardando a origem (*state.de*); com tipo incorreto (ex.: paciente tentando acessar */admin*), redireciona para o próprio dashboard. Esse comportamento foi testado manualmente e confirmado em funcionamento.

Conclusão

Esta atividade revisou criticamente o Clique Saúde v1, propôs um roadmap realista de evolução e entregou um protótipo funcional em React que já implementa a maior parte das fases 1 a 5 do roadmap: componentização com props e children, gerenciamento de estado com hooks, integração com duas APIs REST (pública e a do próprio sistema, replicada fielmente), boas práticas de UX/UI corrigindo os problemas identificados na análise heurística, e um sistema completo de rotas — incluindo proteção por autenticação e por tipo de usuário. A base está pronta para a fase final: substituir a API simulada pela API Flask real em produção.